The TrakSYS Paradox: How We Used MCP to Bridge the Gap Between Raw Data and Real Answers

The TrakSYS Paradox: Why "Having Data" isn't the Same as "Having Answers"
Research question: How can we enable non-technical factory staff to query complex manufacturing data using natural language, while ensuring accuracy and contextual understanding?

You know what's insane? AI is out here doing predictive maintenance, generative design, quality control, all this futuristic stuff. But ask a factory supervisor a simple question like "Why did the line stop yesterday?" and suddenly we're back in the dark ages. They can't get an answer. Not because the data isn't there oh, it's definitely there but because nobody can actually talk to it.

Here's the reality. Companies invest millions in data collection. Sensors everywhere, systems tracking every movement, every batch, every second. But for the people who actually need that data to make decisions? It might as well be invisible.

Take **AG Solution**. Global industrial automation leader. Big name. Their plants generate terabytes of production data through **TrakSYS** a Manufacturing Execution System tracking every order, batch, and equipment event in real time. Sounds impressive, right? It is. But ask a floor supervisor a simple question like "Why did yesterday's shift have so much downtime?" and watch what happens. They submit a ticket. Wait hours. Get a spreadsheet. Maybe. If they're lucky.

Why? Because making sense of that data requires SQL expertise and deep knowledge of complex database relationships. Technical skills that operational staff you know, the people actually running the factory simply don't have time to master in the heat of a production cycle.

So here's the question that got us thinking: What if that gap just... didn't exist? What if a shift supervisor could ask a question in plain English and get an immediate, accurate answer without needing to become a database expert?

That's exactly what we set out to build. This article walks you through how we used the **Model Context Protocol (MCP)** to create an AI assistant that bridges the gap between everyday language and industrial data. We'll show you the architecture, the

tools we built, and how it all comes together to let anyone ask questions and get real answers without exposing sensitive company data or breaking anything. Let's dive in.

Understanding the Data Landscape: More Than Just Tables

So after establishing why this problem matters, guess what our first task was? Yep actually building the MCP server that would solve it. Which meant we had to get up close and personal with a system that, six weeks ago, I'd never even heard of a Manufacturing Execution System, or MES.

Now, I know some of you are gonna be wondering what that is or maybe some of us already know but for those who don't know or didn't know like me six weeks ago? It's basically a system that monitors, tracks, documents, and controls the process of manufacturing goods from raw materials to finished products. Yeah, you heard that right, an "MES".

So with that out the way, let me explain the data structure of the MES we worked with. **TrakSYS** which is the **MES** we used basically tracks production at a granular level. And its data model? It's built around four main layers.

First, production orders are stored in a table called tJob. Weird, right? I thought it'd be named tProduct or something, but nope. This table basically represents a production order or work order from the ERP system. So it contains things like productID and plannedStart basically when you should start, or I suppose when production would start.

Second, batches. But not the kind you bake! Batches are the physical execution of a job. So while the production table defines what needs to be done from the ERP, the batches show what is currently happening or has already happened. Each batch can be seen as a real, trackable instance of production where materials are processed, machines are involved, and actual output is generated.

Third, events. These are the heartbeat of the factory—the specific activities that happen during a batch. For the data engineers in the room, you know the drill: this data isn't in one neat pile. It’s distributed across several specialized tables:

tBatchStep - stores every process step executed with start/end times.
tBatchParameter - stores real-time readings like temperature, filling weight, pressure, etc., with min/max limits
tMaterialUseActual - stores actual raw materials consumed, linked via jobID
DBR - stores free-text operator remarks and notes
tTask - stores mandatory compliance quality tasks and their pass/fail status

Alright, so that's the structure. But here's where things get spicy.

The biggest challenge and I mean biggest when working with an **MES** system is that even something as simple or easy as downtime? It's gonna be difficult. Because everything is spread across 50+ interconnected tables with foreign keys that don't actually have constraints. What I mean is: yeah, an ID can be a foreign key in another table, but there's no formal foreign key constraint tying them together. You'll always have to join multiple tables to get a complete picture of what you're looking for.

This poses another question or rather, more of a curiosity like, how is database access alone able to solve this? Or is raw database access not enough for an AI to solve this?

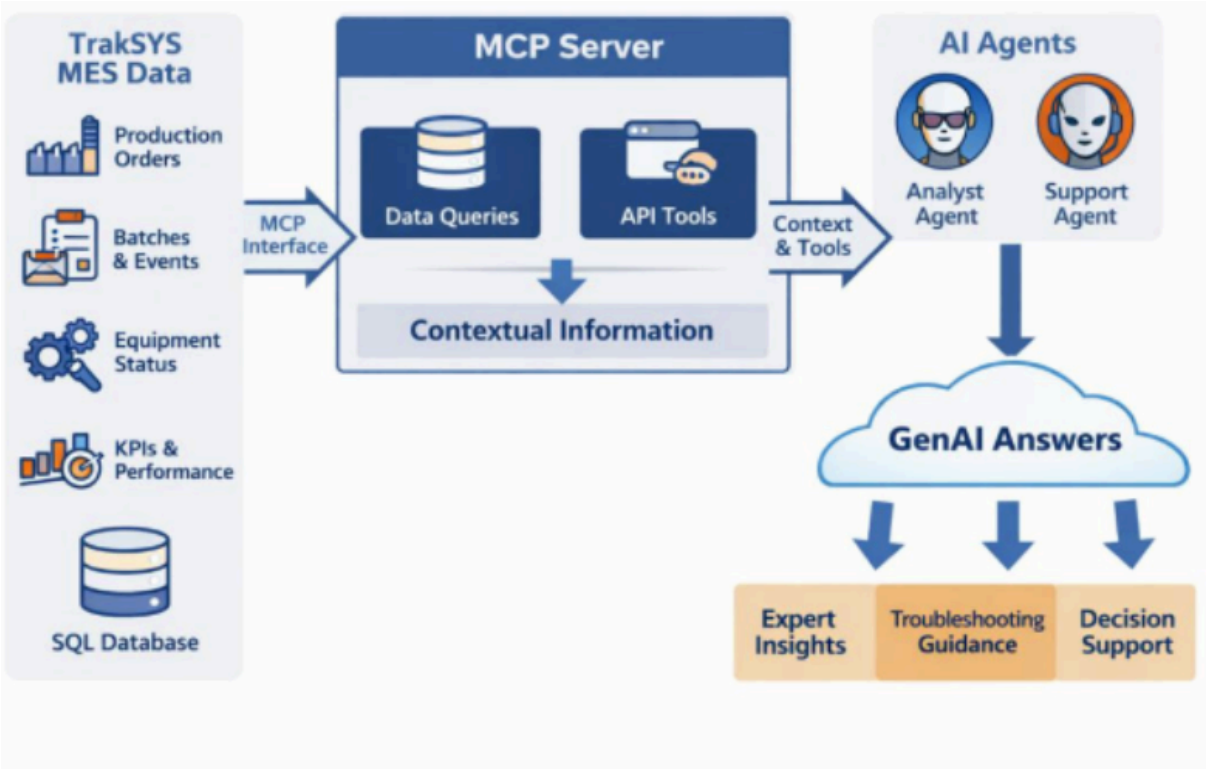
The answer is no. When you just send this to ChatGPT or other LLMs and I can already hear some of you saying "but that's what prompts are for" the thing is, yes and no.

When an AI looks at raw SQL, it just sees column names and numbers. That's not enough context. I mean, imagine asking: what is a job? Or what does a batch actually mean? All of this can't be understood by prompt alone. Which is why the AI or LLM needs to know the exact table relationships to be able to answer your question.

So if raw database access isn't the answer, what is? Let me show you what we built instead and honestly, this is where things get really cool.

The MCP Architecture: A Clean Bridge Between AI and the Factory

Okay, so now we'll look into the MCP architecture, which I think is the most significant technical innovation in this project. It's just a clean, production-grade bridge between an LLM and an MES database.



The MCP architecture as seen in the above image isn't complex to understand—at least at an architectural level. But let me break down what we actually built.

For our implementation, we built three distinct layers:

Data Queries Layer

This layer contains pure, reusable, highly optimized SQL functions with:

- Automatic batch_name → batch_id → job_id resolution (because who wants to do that manually?).
- Intelligent time-window fallback via DataAvailabilityCache.
- Deviation detection, planned vs. actual BOM aggregation (in Python).
- All queries are parameterized and read-only (safety first, people).

API Tools Layer

Each tool is defined using:

- **@mcp.tool()** decorator
- Rich, LLM-friendly descriptions
- Pydantic input_model (strict validation + field descriptions)
- ToolResponse wrapper (success / partial / no_data / error)

Contextual Information Output Layer

This is the secret sauce.

Every tool returns not just raw data, but:

- methodology block (how the answer was derived)
- time_info with fallback explanation

- Business-level signals (is_deviation, total_quality_signals, compliance_rate_pct, etc.)
- Actionable suggestions

Now, when we look at all this MCP, LLM, all that it must've sparked some curiosity. I mean, I'm pretty sure some people are wondering: okay, what makes MCP so good beyond just seeing the architecture? How does it fair against traditional methods? And why do I think this is something everyone needs to try or hop on?

Let me break it down.

Raw SQL is basically useless here. The AI gets little to no business context it just sees table names and numbers. And let's not talk about maintainability, because that's an absolute nightmare. Zero natural language support.

Traditional REST APIs are better on maintainability, sure. But they still have very limited natural language support and low business context. They treat every request as independent, which isn't how conversations work.

MCP, compared to both, excels in every aspect. AI understanding? You give it an incredible amount of business context. Maintainability? Excellent. Natural language support? Great.

In summary, MCP gives the LLM first-class business tools instead of tables, while still giving developers full control and observability.
This is an example of a tool definition:

```python
@self.mcp.tool(  edobor Aisosa
    name="get_batches",
    description="""Query production batches with flexible filtering and time intelligence."""
)
async def get_batches(params: GetBatchesInput) -> dict:
    inputs = params.model_dump()
    async with self.tracing.trace_tool("get_batches", inputs) as span:
        try:
            result = await batches.get_batches(
                time_service=self.time_service, trace_span=span, **inputs,
            )
            response = _build_response(
                data=result["batches"],
                time_info=result.get("time_info"),
                no_data_suggestions=[
                    "Try a broader time_window like 'last 30 days'",
                    "Verify the batch_name or batch_id is correct",
                    "Check if the system_id matches a valid production line",
                ],
            )
            self.tracing.set_output(span, response.to_dict())
            return response.to_dict()
        except Exception as e:
            self.tracing.record_error(span, e,  tool_name: "get_batches")
            self.logger.error( msg: "get_batches failed: %s",  *args: e, exc_info=True)
            return ToolResponse.error(
                message=str(e),
                suggestions=[
                    "Check the database connection",
                    "Verify time_window format (e.g., 'last 7 days')",
                    "Try with just a batch_name or batch_id",
                ],
            ).to_dict()
```

This clean separation SQL abstraction → validated tool → contextual response is what makes the entire system reliable, explainable, and truly LLM-native.

Okay, I know we've read about enough about MCP right about now. But what if I told you that you haven't even seen the best part yet? At least for me.

The Translation Layer: Turning Complexity into Simplicity

I can wholeheartedly say that my favorite part of this project is the translation layer. It literally turns 427 interconnected tables—like, damn, who could go through that? Definitely not me—where the connections are intertwined and sometimes vague, into clean, reliable, business-meaningful tools the LLM can actually use.

The Problem with Raw SQL is that it's Useless to an LLM

A single "simple" question like: "Show me material consumption for batch 456" requires joining at least four tables:

- **tBatch** → tJob (to find the JobID)

- **tMaterialUseActual** (via JobID)
- **tMaterial** (to get names and codes)
- Optional **_SAPBOM** for planned quantities

An LLM trying to write this SQL directly would fail 9 times out of 10. No joke.

The Solution: The Translation Layer
We built a clean abstraction that does three things:

- Hides the complexity (SQL joins, name resolution, time fallbacks).
- Enforces business rules (validation, aliases, safe limits).
- Adds context (methodology, explanations, fallback messages).

Real Example: Time Windows + Filters + Relationships
Here's how the get_batch_materials tool handles a natural question.

If a user asks, for example: "Which materials were consumed for job 1187?"

```
# 1. Name resolution (tBatch.Name → ID)
if batch_name:
    batch_id = await _resolve_batch_name(batch_name)

# 2. Batch → Job relationship (critical!)
if batch_id and not job_id:
    job_id = await _resolve_batch_to_job(batch_id)

# 3. Intelligent time handling
if time_window:
    resolution = await time_service.resolve(time_window, table="tMaterialUseActual")
    # fallback_triggered = True if outside available range
```

The LLM doesn't and won't ever see any of this. It just calls one clean tool. Beautiful, right?

Validation Strategies That Prevent Hallucination

To stop the LLM from guessing or hallucinating on manufacturing data, we built multiple layers of protection directly into the tools:

Pydantic Input Models enforce strict type checking and required fields, while also providing rich, human-readable descriptions that guide the model on how to use each parameter correctly.

Smart Alias Mapping automatically normalizes common user terms for example, converting "Temperature" or "temp" into the exact database name "Temperature_batch," and "Filling Weight" into its proper variant eliminating the most frequent cause of empty results.

ToolResponse Wrapper ensures every answer follows a consistent structure (success, partial, no_data, or error) and never returns raw database rows without context or explanation.

Methodology Field is included in every response so the LLM can see exactly how the data was retrieved, which tables were used, and what business rules were applied—removing any temptation to invent explanations.

Unicode Cleanup fixes encoding issues that appear in some TrakSYS fields, decoding them cleanly before they reach the LLM.

Together, these layers turn a complex, error-prone database into a safe, reliable interface that the AI can trust.
Alright, so we've got this powerful MCP server with all these carefully crafted tools. But a tool is useless without someone who knows how to use it, right? That's where the final piece comes in.

AI Agent Integration: From Raw Data to Expert Answers

And finally, the missing piece: our custom agent. This is the part that actually consumes the MCP tools we built. This is where it all happens. This is where raw data turns into expert-level manufacturing? answers.

How the Agent Consumes MCP Tools

The agent follows a clean and reliable loop:

So here's how it works: the agent uses **OpenAI Function Calling**, where the LLM decides which tool(s) to call based on the user question. Those tools are made available through a **Function Registry** that dynamically converts every MCP tool into an OpenAI-compatible format with name, description, and parameters. When multiple tools need to be called, the agent handles them in **Parallel Execution** using asyncio.gather for speed. As answers come back, the agent delivers them through **Streaming Response** back to OpenWebUI in real time. And every result from the MCP server is wrapped in a **ToolResponse Wrapper** with status, methodology, time_info, and suggestions before being sent back to the LLM.

What does all that mean? It means this architecture allows the agent to reason step-by-step, call tools in parallel, and explain its thinking exactly like a senior manufacturing engineer. Pretty cool, right?

[Demo Scenario: "Why did Line 3 stop yesterday?"]

Let me show you what this actually looks like in action. Here's the real internal flow when a user asks that question:

User: "Why did Line 3 stop yesterday?"

Agent Orchestrator (Step-by-step reasoning):

- Calls get_batches with system_name="E3" + time_window="yesterday" → finds relevant batches
- In parallel, calls calculate_oee for Line 3 on the same date → gets Availability %
- Also calls get_oee_downtime_events for Line 3 → returns actual downtime events with fault codes and operator notes
- Finally calls get_batch_quality_analysis on the affected batches → checks for parameter deviations and incomplete tasks

Final Answer (what the user sees):

- "Line 3 stopped for 87 minutes yesterday (Availability = 68%).
  Main causes:
- Equipment fault "E3-FAULT-12" from 09:14–10:41 (operator noted "conveyor jam")
- One incomplete compliance task (Weighing Checklist)
- No parameter deviations recorded.
  This single event dropped the daily OEE by ~22 points."

Why did line E2 Stop?

TrakSYS-Agent Today at 1:38 AM

🔎 Data Retrieval:

- Queried the current status of line E2 using the `get_equipment_state` tool with active fault checking enabled.

🧠 Reasoning / Method:

- Checked if there are any active faults or downtime events currently affecting line E2.
- The tool returned the current status of line E2, indicating whether it is running or faulted.

✅ Final Answer:

- Line E2 is currently not running, but there are no active faults reported. This suggests that the line might be stopped due to reasons other than equipment faults, such as scheduled downtime or manual intervention.

Calculate oee for line E2?

TrakSYS-Agent Today at 1:44 AM

Data Retrieval:

- Calculated OEE for line E2 from March 10 to March 12, 2026, using the `calculate_oee` tool.

Reasoning / Method:

- The OEE calculation considers three components: Availability, Performance, and Quality. Each day's OEE is computed based on the actual operating time, performance ratios, and quality metrics. This provides a comprehensive view of the line's efficiency.

Final Answer:

- **March 10, 2026:** OEE was 71.88% with Availability at 84.72%, Performance at 86.07%, and Quality at 98.57%.
- **March 11, 2026:** OEE was 84.38% with Availability at 93.75%, Performance at 91.11%, and Quality at 98.78%.
- **March 12, 2026:** OEE was 93.75% with Availability at 98.44%, Performance at 96.3%, and Quality at 98.9%.

These figures indicate a consistent improvement in line efficiency over the three days.

Can you show me the downtime events for line E2 during this period?

TrakSYS-Agent

Data Retrieval:

- Retrieved downtime events for line E2 from March 10 to March 12, 2026, using the `get_oee_downtime_events` tool.

Reasoning / Method:

- Analyzed the events that occurred on line E2 during the specified period, focusing on the impact and descriptions to understand the reasons for downtime.

Final Answer:

During the period from March 10 to March 12, 2026, line E2 experienced the following downtime events:

1. **Sensor Failure:**
   - **Date/Time:** March 12, 2026, from 21:10 to 21:25
   - **Impact:** 900 seconds
   - **Description:** Optical sensor lost alignment.
   - **Notes:** Optical sensor PE-404 lost alignment.
2. **Infeed Jam:**
   - **Date/Time:** March 12, 2026, from 15:30 to 15:55
   - **Impact:** 1500 seconds
   - **Description:** Severe jam at the infeed conveyor.
   - **Notes:** Required manual clearing.
3. **Motor Fault:**
   - **Date/Time:** March 12, 2026, from 07:15 to 07:35
   - **Impact:** 1200 seconds
   - **Description:** Main drive motor overload tripped.
   - **Notes:** Main drive motor overload tripped.

These events contributed to the downtime and affected the line's overall efficiency.

Send a Message

See what happened there? The agent didn't guess—it used four tools in parallel and combined the results with business context. It's like having a manufacturing expert on call 24/7.

How We Ensured Expert-Level Accuracy

- Rich tool descriptions in the MCP layer guide the LLM on when and how to use each tool.

- Methodology field in every response explains exactly how the answer was calculated.
- Parallel tool calling prevents the agent from missing important signals.
- System prompt contains deep domain knowledge (product codes, material units, batch states, known data gaps, etc.).
- Fallback handling is explicit—the agent always tells the user when it used fallback data.
- No raw SQL exposure the LLM never sees table names or joins.

This combination turns a normal LLM into a true manufacturing expert that can diagnose issues, explain root causes, and suggest next actions with confidence.

Conclusion

So after all that the 427 tables, the complex joins, the translation layer, the custom agent where did we land?

Honestly? We built something pretty cool. Our MCP server and custom AI agent now let non-technical staff ask questions in plain English and get instant, accurate answers from TrakSYS data. Batches, events, KPIs all of it. No tickets. No waiting. No SQL required.

What We Achieved
MCP gave us something raw SQL and REST APIs couldn't: validated tools with real business context. The agent runs parallel tool calls, explains its reasoning, and delivers expert-level analysis. That whole "Why did Line 3 stop?" example? That's not a demo that's the actual system working.

What Almost Broke Us
Let me be real with you. We came from a world where databases had maybe 50 tables. Then we walked into 427 tables with no foreign key constraints. Yeah. We spent more time figuring out what the data meant than writing code. Because here's the thing the MCP server and agent are only as good as the context you give them. And tracing relationships across a database that refuses to connect itself? That was the hardest part.

What We Learned
Not all LLMs are built the same. Some gave great answers. Some gave... creative ones. That's why we built our own agent with custom context. Also, traceability tools? Surprisingly lacking. We ended up building what we needed because nothing off the

shelf fit our industrial requirements. And one more thing: data quality matters. Garbage in, garbage out—AI doesn't magically fix bad data.

What's Next
More tools. Better context handling. A UI that anyone can use without training. The foundation is there.

The Real Impact
Here's what this means: what used to take hours—tickets, waiting, spreadsheets—now takes seconds. A supervisor asks a question. The system answers. Decision made. Production keeps moving. That's not just efficiency. That's empowering the people who actually run the factory.

So Here's My Pitch
Whether you're in manufacturing or any industry dealing with complex data, give MCP a shot. It turns data into conversation. It turns "I don't know" into "here's exactly what happened." And honestly? It's just really satisfying to watch someone ask a question in plain English and get an expert-level answer back in seconds.

Try it out.

REFERENCES

AI in manufacturing: A comprehensive guide | SAP https://www.sap.com/products/scm/digital-manufacturing/what-is-mes.html

What is the Model Context Protocol (MCP)? - Model Context Protocol